

Computação Gráfica e Processamento de Imagem



Professor: EDILSON LIMA

ATIVIDADE PRÁTICA – SÍNTESE DE IMAGEM

Atividade Prática: Simulação de Ray Casting em Python

Objetivo

Implementar uma simulação simplificada do algoritmo de **ray casting** para identificar qual triângulo de uma cena 2D é visível a partir de um ponto de projeção (câmera), considerando oclusão.

Conteúdo Relacionado

- Pipeline tradicional de visualização
- Transformações geométricas
- Reflexão difusa
- Oclusão e visibilidade
- Ray casting

Ferramentas

- Google Colab
- Bibliotecas: numpy, matplotlib

Etapas da Atividade

1. Modelagem da Cena

- Criar dois triângulos em 2D com vértices definidos.
- Definir o ponto de projeção (CA) e o ponto de observação (P).

```
import numpy as np
import matplotlib.pyplot as plt

# Triângulos definidos por seus vértices
tri1 = np.array([[1, 3], [2, 5], [3, 3]])
tri2 = np.array([[2, 2], [3, 4], [4, 2]])

# Centro de projeção
CA = np.array([0, 0])
P = np.array([3, 3])
```

2. Visualização da Cena

- Plotar os triângulos, o ponto CA e a linha de projeção.

```
def plot_scene():
    plt.figure(figsize=(6,6))
    plt.fill(tri1[:,0], tri1[:,1], 'orange', alpha=0.5, label='Triângulo 1')
    plt.fill(tri2[:,0], tri2[:,1], 'blue', alpha=0.5, label='Triângulo 2')
    plt.plot([CA[0], P[0]], [CA[1], P[1]], 'k--', label='Raio projetado')
    plt.scatter(*CA, color='black', label='CA')
    plt.scatter(*P, color='red', label='Ponto P')
    plt.legend()
    plt.grid(True)
    plt.title('Cena 2D com Ray Casting')
    plt.show()
```

```
plot_scene()
```

3. Interseção com Triângulos

- Implementar função para verificar se o raio intersecta algum triângulo.
- Determinar qual triângulo é o primeiro intersectado.

```
from matplotlib.path import Path
```

```
def ray_intersects(triangle, CA, P):
    path = Path(triangle)
    direction = P - CA
    for t in np.linspace(0, 1, 100):
```

```

    point = CA + t * direction
    if path.contains_point(point):
        return True, point
    return False, None

# Verificar interseções
inter1, pt1 = ray_intersects(tri1, CA, P)
inter2, pt2 = ray_intersects(tri2, CA, P)

# Determinar o triângulo visível
if inter1 and inter2:
    d1 = np.linalg.norm(pt1 - CA)
    d2 = np.linalg.norm(pt2 - CA)
    visible = 'Triângulo 1' if d1 < d2 else 'Triângulo 2'
elif inter1:
    visible = 'Triângulo 1'
elif inter2:
    visible = 'Triângulo 2'
else:
    visible = 'Nenhum triângulo intersectado'

print(f'Triângulo visível: {visible}')

```

Desafio Extra (obrigatório)

- Adicionar mais triângulos à cena.
- Implementar sombras (bloqueio de luz).
- Simular reflexão indireta com múltiplos raios.

Entrega

- Notebook com código funcional.
- Comentários explicativos sobre cada etapa.
- Print da cena renderizada com identificação do triângulo visível.